



DocuProcess

Intelligent Document Pipeline

AWS Academy Learner Lab — Step-by-Step Implementation Manual

WOA7016 Cloud Computing Capstone Project

Group 7 • Client: JusticeArch Legal Firm

SOLUTION OVERVIEW
This manual guides you through building a fully automated, event-driven document
ingestion pipeline on AWS. Scanned PDF invoices and contracts uploaded to S3 are
automatically processed by Textract via Lambda. Extracted metadata is stored in
DynamoDB. A human-review portal on EC2 flags low-confidence results (<80%) for
staff validation. Processed files are archived automatically after 30 days.

Table of Contents

Phase 0: Prerequisites & Account Setup	3
Phase 1: Custom VPC & Networking	4
Phase 2: S3 Buckets & Lifecycle Rules	5
Phase 3: DynamoDB Table	6
Phase 4: IAM Roles & Least Privilege	7
Phase 5: Lambda Function (Textract Processor)	8
Phase 6: S3 Event Notification (Trigger)	10
Phase 7: EC2 Review Portal Setup	11
Phase 8: Auto Scaling Group & Application Load Balancer	14
Phase 9: Security Groups & VPC Hardening	16
Phase 10: End-to-End Testing	17
Phase 11: Advanced Challenge — Keyword Search	18
Phase 12: Resource Tagging & Submission Checklist	19

Phase 0: Prerequisites & Account Setup

0.1 Log in to AWS Academy Learner Lab

1. Go to <https://awsacademy.instructure.com> and log in with your student credentials.
2. Navigate to your course and click "Modules," then find the Learner Lab module.
3. Click "Start Lab." Wait until the circle next to "AWS" turns green (can take 1–2 minutes).
4. Click the green "AWS" button to open the AWS Management Console in a new tab.

 **Warning:** *Learner Lab sessions expire after 4 hours. Save your work frequently. Resources persist between sessions, but you must restart the lab each time.*

0.2 Set Your Working Region

5. In the AWS Console top-right, verify the region is set to US East (N. Virginia) — us-east-1.
6. If not, click the region name and select "US East (N. Virginia)."

 **Note:** *All services in this guide must be created in the same region. us-east-1 is recommended as it supports all required services.*

0.3 Verify Service Availability

Confirm you have access to these services (search each in the Console search bar):

- Amazon S3
- Amazon DynamoDB
- AWS Lambda
- Amazon Textract
- Amazon EC2
- Amazon VPC
- AWS IAM

Phase 1: Custom VPC & Networking

 **Note:** Do NOT use the Default VPC. Create a custom VPC to meet the mandatory technical standards.

1.1 Create the VPC

7. In the Console search bar, type VPC and click VPC.
8. Click "Your VPCs" in the left sidebar, then click "Create VPC."
9. Select "VPC only."
10. Fill in the details:

Field	Value
Name tag	justicearch-vpc
IPv4 CIDR block	10.0.0.0/16
IPv6 CIDR block	No IPv6 CIDR block
Tenancy	Default

11. Click "Create VPC."

1.2 Create Subnets

Create 4 subnets: 2 public and 2 private, across 2 Availability Zones for high availability.

12. In the VPC Dashboard, click "Subnets" → "Create subnet."
13. Select VPC: justicearch-vpc.
14. Create the following subnets one by one (click Add new subnet between each):

Subnet Name	Availability Zone	CIDR Block
justicearch-public-1a	us-east-1a	10.0.1.0/24
justicearch-public-1b	us-east-1b	10.0.2.0/24
justicearch-private-1a	us-east-1a	10.0.3.0/24
justicearch-private-1b	us-east-1b	10.0.4.0/24

15. Click "Create subnet."

1.3 Enable Auto-Assign Public IP on Public Subnets

16. Select "justicearch-public-1a," click "Actions" → "Edit subnet settings."
17. Check "Enable auto-assign public IPv4 address." Click Save.
18. Repeat for justicearch-public-1b.

1.4 Create and Attach Internet Gateway

19. Click "Internet Gateways" → "Create internet gateway."
20. Name: justicearch-igw. Click "Create internet gateway."
21. Click "Actions" → "Attach to VPC." Select justicearch-vpc. Click "Attach."

1.5 Configure Public Route Table

22. Click "Route Tables" → "Create route table."
23. Name: justicearch-public-rt. VPC: justicearch-vpc. Click Create.
24. Select the new route table. Click the Routes tab → Edit routes.
25. Click "Add route": Destination = 0.0.0.0/0, Target = Internet Gateway → justicearch-igw. Save.
26. Click the Subnet associations tab → Edit subnet associations.
27. Select both public subnets. Save.

1.6 Create NAT Gateway (for Private Subnet Internet Access)

28. Click "NAT Gateways" → "Create NAT gateway."
29. Name: justicearch-nat. Subnet: justicearch-public-1a. Connectivity: Public.
30. Click "Allocate Elastic IP," then click "Create NAT gateway."
31. Create a private route table: Name = justicearch-private-rt, VPC = justicearch-vpc.
32. Add route: 0.0.0.0/0 → NAT Gateway → justicearch-nat.
33. Associate both private subnets with this route table.

Phase 2: S3 Buckets & Lifecycle Rules

2.1 Create the Inbox Bucket

34. Search for "S3" in the Console. Click "Create bucket."
35. Configure as follows:

Field	Value
Bucket name	justicearch-inbox-[yourname] (must be globally unique)
AWS Region	US East (N. Virginia) us-east-1
Block Public Access	Keep all defaults (block everything)
Versioning	Disabled (for now)
Tags	Project: Capstone, Group: 7

36. Click "Create bucket."

2.2 Create a Processed Prefix (Simulated Folder)

37. Open the bucket. Click "Create folder."
38. Name the folder: processed. Click "Create folder."

 **Note:** S3 doesn't have real folders; prefixes simulate them. Uploaded PDFs go to the root. The Lifecycle Rule will move them to the processed/ prefix after 30 days.

2.3 Add Lifecycle Rule to Archive Processed Files

39. In your bucket, click the "Management" tab → "Create lifecycle rule."
40. Fill in:
 - Rule name: archive-processed-30days
 - Prefix: processed/
41. Under Lifecycle rule actions, check: Transition current versions of objects between storage classes.
42. Choose storage class: S3 Glacier Flexible Retrieval. Days after object creation: 30.
43. Click "Create rule."

 **Tip:** This satisfies the Archival requirement. Files tagged with the processed/ prefix automatically move to Glacier-class cold storage after 30 days, reducing costs.

Phase 3: DynamoDB Table

3.1 Create the Documents Table

44. Search "DynamoDB" in the Console. Click "Create table."

45. Configure:

Field	Value
Table name	JusticeArchDocuments
Partition key	documentId (String)
Sort key	(leave blank)
Table settings	Customize settings
Table class	DynamoDB Standard
Read/write capacity	On-demand (pay-per-use — best for Learner Lab)

46. Add tags: Project: Capstone, Group: 7.

47. Click "Create table." Wait for status to show Active.

3.2 Understand the Data Schema

Each DynamoDB record will store the following attributes (Lambda writes these):

Attribute	Description
documentId	UUID generated per document (Partition Key)
s3Key	S3 object path (e.g., invoices/file.pdf)
vendor	Extracted vendor name from Textract
date	Extracted document date
amount	Extracted dollar amount
confidence	Average Textract confidence score (0–100)
flaggedForReview	Boolean — true if confidence < 80
status	PENDING REVIEWED APPROVED
uploadedAt	ISO timestamp of upload

Phase 4: IAM Roles & Least Privilege

 **Note:** *The Learner Lab restricts IAM role creation. You will use the pre-created LabRole for Lambda. For EC2, attach an instance profile.*

4.1 Confirm the LabRole Exists

48. Search "IAM" in the Console. Click "Roles."
49. Search for "LabRole." Click on it.
50. Note the ARN — you will use this when creating the Lambda function.
51. Verify it has the following managed policies attached:
 - AmazonS3FullAccess
 - AmazonDynamoDBFullAccess
 - AmazonTextractFullAccess
 - AWSLambdaBasicExecutionRole

 **Tip:** *In production, you would create a custom policy with exactly the permissions needed. For the Learner Lab, LabRole has broad permissions pre-configured.*

4.2 Create an EC2 Instance Profile for DynamoDB Access

52. In IAM, click "Roles" → "Create role."
53. Trusted entity: AWS service. Use case: EC2. Click Next.
54. Search for and attach these policies:
 - AmazonDynamoDBFullAccess
 - AmazonS3ReadOnlyAccess
55. Name the role: justicearch-ec2-role. Add tags. Click Create role.

Phase 5: Lambda Function (Textract Processor)

5.1 Create the Lambda Function

56. Search "Lambda" in the Console. Click "Create function."
57. Select "Author from scratch."
58. Configure:

Field	Value
Function name	JusticeArchProcessor
Runtime	Python 3.12
Architecture	x86_64
Execution role	Use an existing role → LabRole

59. Click "Create function."

5.2 Configure Lambda Settings

60. In the function, click "Configuration" tab → "General configuration" → Edit.
61. Set:
 - Memory: 512 MB
 - Timeout: 2 minutes 0 seconds (Textract can take time for large PDFs)
62. Click Save.

5.3 Add the Lambda Function Code

63. Click the "Code" tab. In the code editor, replace the default code with the following:

Paste the code below into the Lambda inline code editor (lambda_function.py):

```
import json, boto3, uuid, datetime

s3 = boto3.client('s3')
textract = boto3.client('textract')
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('JusticeArchDocuments')

def lambda_handler(event, context):
```

```

record    = event['Records'][0]
bucket    = record['s3']['bucket']['name']
key       = record['s3']['object']['key']

# Call Textract to extract key-value pairs
response = textract.analyze_document(
    Document={'S3Object': {'Bucket': bucket, 'Name': key}},
    FeatureTypes=['FORMS']
)

blocks    = response.get('Blocks', [])
vendor    = ''
date      = ''
amount    = ''
scores    = []

for block in blocks:
    if block['BlockType'] == 'KEY_VALUE_SET':
        conf = block.get('Confidence', 0)
        scores.append(conf)
        if 'Relationships' in block:
            text = block.get('Text', '').lower()
            if 'vendor' in text: vendor = text
            if 'date' in text: date = text
            if 'amount' in text or 'total' in text: amount = text

avg_confidence = sum(scores) / len(scores) if scores else 0
flagged = avg_confidence < 80.0

doc_id = str(uuid.uuid4())
table.put_item(Item={
    'documentId': doc_id,
    's3Key': key,
    'vendor': vendor or 'UNKNOWN',
    'date': date or 'UNKNOWN',
    'amount': amount or 'UNKNOWN',
    'confidence': str(round(avg_confidence, 2)),
    'flaggedForReview': flagged,
    'status': 'PENDING',
    'uploadedAt': datetime.datetime.utcnow().isoformat()
})

print(f'Processed {key}: confidence={avg_confidence:.1f}%, flagged={flagged}')
return {'statusCode': 200, 'body': json.dumps('Done')}

```

64. Click "Deploy" (orange button) to save and deploy the function.

Phase 6: S3 Event Notification (Trigger)

6.1 Grant S3 Permission to Invoke Lambda

65. Open the Lambda function "JusticeArchProcessor."

66. Click "Configuration" → "Permissions" → scroll to "Resource-based policy statements."

67. Click "Add permissions."

68. Fill in:

Field	Value
Policy type	AWS service
Service	S3
Statement ID	S3InvokePermission
Principal	s3.amazonaws.com
Source ARN	arn:aws:s3:::justicearch-inbox-[yourname]
Action	lambda:InvokeFunction

69. Click "Save."

6.2 Configure the S3 Event Notification

70. Open your S3 bucket (justicearch-inbox-[yourname]).

71. Click "Properties" tab. Scroll to "Event notifications." Click "Create event notification."

72. Fill in:

Field	Value
Event name	NewPDFUpload
Prefix (optional)	(leave blank to trigger on all uploads)
Suffix	.pdf
Event types	✓ s3:ObjectCreated:Put
Destination	Lambda function
Lambda function	JusticeArchProcessor

73. Click "Save changes."

 **Tip:** The suffix *.pdf* ensures only PDF uploads trigger the Lambda — other file types are ignored.

Phase 7: EC2 Review Portal Setup

7.1 Create a Security Group for the EC2 Instance

74. Search "EC2" in the Console. In the left sidebar, click "Security Groups" → "Create security group."

75. Configure:

Field	Value
Security group name	justicearch-ec2-sg
Description	EC2 Review Portal Security Group
VPC	justicearch-vpc

76. Add Inbound Rules:

Type	Port	Source
SSH	22	Your IP (e.g., 203.0.113.0/32)
HTTP	80	0.0.0.0/0 (ALB will front-end this)
Custom TCP	5000	0.0.0.0/0 (Flask app port)

77. Click "Create security group."

7.2 Launch the EC2 Instance

78. In EC2, click "Instances" → "Launch instances."

79. Configure:

Field	Value
Name	JusticeArch-ReviewPortal
AMI	Amazon Linux 2023 AMI (free tier eligible)
Instance type	t2.micro (free tier)
Key pair	Create new key pair: justicearch-key (RSA, .pem)
VPC	justicearch-vpc
Subnet	justicearch-public-1a
Auto-assign Public IP	Enable
Security groups	Select existing: justicearch-ec2-sg
IAM instance profile	justicearch-ec2-role
Tags	Project: Capstone, Group: 7, Name: ReviewPortal

80. Click "Launch instance."

7.3 Connect to the EC2 Instance

81. Wait 2–3 minutes for the instance to reach Running state.
82. Select the instance. Click "Connect" → "EC2 Instance Connect."
83. Click "Connect." A browser-based terminal opens.

 **Note:** Alternatively, connect via SSH: `ssh -i justicearch-key.pem ec2-user@<Public-IP>`

7.4 Install and Configure the Review App

Run these commands in the EC2 terminal in order:

```
# Update system packages
sudo dnf update -y

# Install Python 3 and pip
sudo dnf install python3 python3-pip -y

# Install Flask and boto3
pip3 install flask boto3

# Create the app directory
mkdir /home/ec2-user/review-app && cd /home/ec2-user/review-app

# Create the Flask app file
cat > app.py << 'EOF'
from flask import Flask, render_template_string, request, redirect
import boto3

app = Flask(__name__)
dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
table = dynamodb.Table('JusticeArchDocuments')

TEMPLATE = '''
<!DOCTYPE html><html><head><title>JusticeArch Review Portal</title>
<style>body{font-family:Arial;margin:40px;background:#f4f8fb}
table{border-collapse:collapse;width:100%}
th{background:#1F4E79;color:white;padding:10px}
td{border:1px solid #ccc;padding:8px}
tr:nth-child(even){background:#eaf1fb}
.flags{color:red;font-weight:bold}</style></head>
<body><h1>JusticeArch Document Review Portal</h1>
<form method=GET action=/search>
```

```

<input name=q placeholder='Search keyword...'> <button>Search</button></form>
<br><table><tr><th>ID</th><th>S3 Key</th><th>Vendor</th>
<th>Date</th><th>Amount</th><th>Confidence</th><th>Status</th><th>Action</th></tr>
{% for doc in docs %}<tr>
<td>{{ doc.documentId[:8] }}</td><td>{{ doc.s3Key }}</td>
<td>{{ doc.vendor }}</td><td>{{ doc.date }}</td><td>{{ doc.amount }}</td>
<td class='{{ "flag" if doc.flaggedForReview else "" }}'>
{{ doc.confidence }}%</td><td>{{ doc.status }}</td>
<td><form method=POST action=/approve>
<input type=hidden name=id value={{ doc.documentId }}>
<button>Approve</button></form></td></tr>{% endfor %}
</table></body></html>'''

```

```

@app.route('/')
def index():
    result = table.scan(
        FilterExpression='flaggedForReview = :t',
        ExpressionAttributeValues={':t': True}
    )
    return render_template_string(TEMPLATE, docs=result['Items'])

```

```

@app.route('/search')
def search():
    q = request.args.get('q', '')
    result = table.scan()
    docs = [d for d in result['Items']
            if q.lower() in str(d).lower()]
    return render_template_string(TEMPLATE, docs=docs)

```

```

@app.route('/approve', methods=['POST'])
def approve():
    doc_id = request.form['id']
    table.update_item(
        Key={'documentId': doc_id},
        UpdateExpression='SET #s = :v',
        ExpressionAttributeNames={'#s': 'status'},
        ExpressionAttributeValues={':v': 'APPROVED'}
    )
    return redirect('/')

```

```

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
EOF

```

```

# Run the app
python3 app.py &

```


84. The app runs on port 5000. Access it at <http://<EC2-Public-IP>:5000>

7.5 Make the App Start on Boot (Optional)

```
sudo tee /etc/systemd/system/reviewapp.service << EOF
[Unit]
Description=JusticeArch Review App
[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/review-app
ExecStart=/usr/bin/python3 app.py
Restart=always
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl enable reviewapp
sudo systemctl start reviewapp
```

Phase 8: Auto Scaling Group & Application Load Balancer

 **Note:** This phase satisfies the mandatory Scalability requirement: ASG + ALB.

8.1 Create a Launch Template

85. In EC2, click "Launch Templates" → "Create launch template."

86. Configure:

Field	Value
Launch template name	justicearch-lt
AMI	Same Amazon Linux 2023 AMI as Phase 7
Instance type	t2.micro
Key pair	justicearch-key
Security groups	justicearch-ec2-sg
IAM instance profile	justicearch-ec2-role

87. Under Advanced details → User data, paste:

```
#!/bin/bash
dnf update -y
dnf install python3 python3-pip -y
pip3 install flask boto3
mkdir /home/ec2-user/review-app
# (Add your app.py content here or pull from S3)
python3 /home/ec2-user/review-app/app.py &
```

88. Add tags. Click Create launch template.

8.2 Create an Application Load Balancer

89. In EC2, click "Load Balancers" → "Create load balancer" → "Application Load Balancer."

90. Configure:

Field	Value
Name	justicearch-alb
Scheme	Internet-facing
IP address type	IPv4
VPC	justicearch-vpc
Availability Zones	Select: public-1a and public-1b subnets

91. Create a Security Group for the ALB:

- Name: justicearch-alb-sg
- Inbound: HTTP port 80 from 0.0.0.0/0

92. Create a Target Group:

- Name: justicearch-tg
- Target type: Instances
- Protocol: HTTP, Port: 5000
- Health check path: /

93. Add listener: HTTP:80 → forward to justicearch-tg.

94. Click "Create load balancer."

8.3 Create the Auto Scaling Group

95. In EC2, click "Auto Scaling Groups" → "Create Auto Scaling group."

96. Configure:

Field	Value
Name	justicearch-asg
Launch template	justicearch-lt
VPC	justicearch-vpc
Availability Zones	Select both public subnets
Load balancing	Attach to an existing load balancer: justicearch-tg
Health check type	ELB
Desired capacity	1
Minimum capacity	1
Maximum capacity	3

97. Scaling policy: Target tracking → CPU utilization target = 70%.

98. Add tags: Project: Capstone, Group: 7.

99. Click "Create Auto Scaling group."

 **Tip:** The ASG will automatically add EC2 instances when CPU exceeds 70%, and remove them when demand drops.

Phase 9: Security Groups & VPC Hardening

9.1 Restrict EC2 Direct Access

Update justicearch-ec2-sg to only accept port 5000 traffic from the ALB security group:

100. Select justicearch-ec2-sg. Click "Inbound rules" → "Edit inbound rules."
101. Change the Custom TCP port 5000 rule:
 - Source: Custom → select justicearch-alb-sg
102. Keep port 22 restricted to your IP. Remove 0.0.0.0/0 from port 5000.
103. Save rules.

9.2 VPC Endpoint for DynamoDB (Best Practice)

Route DynamoDB traffic through AWS internal network instead of the internet:

104. Go to VPC → "Endpoints" → "Create endpoint."
105. Service category: AWS services. Search: DynamoDB. Select com.amazonaws.us-east-1.dynamodb.
106. Type: Gateway. VPC: justicearch-vpc.
107. Route tables: select both private route tables.
108. Click "Create endpoint."

 **Tip:** VPC endpoints improve security (traffic never leaves AWS) and can reduce data transfer costs.

9.3 Security Summary

Component	Security Measure	Standard Met
Lambda	LabRole with minimal required permissions	Least Privilege
EC2	SG allows only ALB → EC2 on port 5000	Defense in Depth
DynamoDB	VPC Gateway Endpoint, IAM role access only	Network Isolation
S3	Block all public access enabled	Data Protection
VPC	Public/private subnet separation	Network Segmentation

Phase 10: End-to-End Testing

10.1 Test the Full Pipeline

109. Obtain a sample PDF (any scanned invoice or document).
110. Open your S3 bucket (justicearch-inbox-[yourname]). Click "Upload" → "Add files."
111. Upload the PDF. Click Upload.

 **Note:** This triggers the Lambda function via the S3 Event Notification configured in Phase 6.

10.2 Verify Lambda Execution

112. Open Lambda → JusticeArchProcessor → "Monitor" tab → "View CloudWatch logs."
113. Open the latest log stream. Look for: "Processed [filename]: confidence=XX%, flagged=True/False."
114. If you see an error, check: IAM permissions, Textract regional availability, S3 trigger config.

10.3 Verify DynamoDB Record

115. Open DynamoDB → JusticeArchDocuments.
116. Click "Explore table items." You should see a new item with your document's metadata.
117. Verify: vendor, date, amount, confidence, flaggedForReview, status fields are populated.

10.4 Verify the Review Portal

118. If flaggedForReview is true, open <http://<EC2-IP>:5000> or the ALB DNS name.
119. You should see the document listed in the portal.
120. Click "Approve" to update the status to APPROVED in DynamoDB.

10.5 Common Errors & Fixes

Error	Likely Cause	Fix
AccessDenied in Lambda logs	IAM role missing Textract permission	Verify LabRole has AmazonTextractFullAccess
Lambda not triggered	S3 event not configured correctly	Re-check Phase 6 — suffix must be .pdf
EC2 app not accessible	Security group blocking port 5000	Re-check Phase 9.1 inbound rules

Error	Likely Cause	Fix
DynamoDB write fails	Wrong table name in Lambda code	Ensure table name matches exactly: JusticeArchDocuments

Phase 11: Advanced Challenge — Keyword Search

The search feature is already included in the Flask app code from Phase 7. This phase explains how it works and how to extend it.

11.1 How the Search Works

The /search route in the Flask app:

- Scans the entire DynamoDB table using `table.scan()`
- Filters results in Python where the query string matches any attribute
- Returns matching documents rendered in the same HTML table

Example: Searching for "Law" returns all records where any field contains "Law." Searching for "Python" would match records where the `s3Key` or extracted text contains "Python."

11.2 Improve Search with DynamoDB GSI (Optional Enhancement)

For faster search on large datasets, create a Global Secondary Index:

121. Open DynamoDB → JusticeArchDocuments → Indexes tab.
122. Click "Create index."
123. Partition key: `vendor` (String). Index name: `vendor-index`.
124. Click Create index. Wait for it to become Active.
125. Update the Flask /search route to use `query()` on the GSI instead of `scan()` for better performance.

11.3 Test the Search

126. Upload several PDFs with different vendors.
127. In the Review Portal, type a vendor name or keyword in the search box.
128. Verify that only matching documents appear.

Phase 12: Resource Tagging & Submission Checklist

12.1 Tag All Resources

Every AWS resource must be tagged. Use the Tag Editor to audit:

129. Search "Tag Editor" in the Console (under Resource Groups & Tag Editor).
130. Select Region: us-east-1. Resource types: All. Click "Search resources."
131. For any resource missing tags, select it and click Manage tags.
132. Add: Project = Capstone, Group = 7.

12.2 Architecture Component Checklist

Requirement	Status
Custom VPC with public & private subnets	☐ Complete
Internet Gateway + NAT Gateway	☐ Complete
S3 Bucket: justicearch-inbox-[name]	☐ Complete
S3 Lifecycle Rule (30-day archive)	☐ Complete
DynamoDB Table: JusticeArchDocuments	☐ Complete
IAM Role: justicearch-ec2-role	☐ Complete
Lambda Function: JusticeArchProcessor	☐ Complete
S3 Event Notification → Lambda	☐ Complete
EC2 Instance: JusticeArch-ReviewPortal	☐ Complete
Flask Review App running on port 5000	☐ Complete
Application Load Balancer (ALB)	☐ Complete
Auto Scaling Group (1–3 instances)	☐ Complete

Requirement	Status
Security Groups (least privilege)	☐ Complete
VPC Endpoint for DynamoDB	☐ Complete
Keyword Search feature on Review Portal	☐ Complete
All resources tagged: Project:Capstone, Group:7	☐ Complete

12.3 Presentation Preparation Tips

- Take screenshots of each component during setup for your slides.
- Record the DynamoDB items before and after a PDF upload to show the pipeline working.
- Show the CloudWatch logs during the live demo to prove Lambda executed.
- Have the ALB DNS name ready to open the Review Portal in real time.
- Prepare a test PDF with clear Vendor/Date/Amount fields for the demo.

12.4 Cost Optimization Notes for Presentation

Service	Cost Strategy
DynamoDB	On-demand billing — pay only for what you use
Lambda	Pay per invocation — free tier: 1M requests/month
S3	Lifecycle rules move files to Glacier (90% cheaper) after 30 days
EC2	t2.micro free tier; ASG scales to 1 instance when idle
Textract	Charged per page processed; ideal for batch workflows

End of Manual — Good Luck, Group 7!

WOA7016 Cloud Computing Capstone • JusticeArch DocuProcess Pipeline